

Composition in WPEWebKit with Skia

Web Engines Hackfest

June 2026, A Coruña

Carlos Garcia Campos

Before Skia

- Layers with tiled backing store
 - Main thread records layer contents into a display list
 - Worker threads replay the display list into a cairo image surface
 - On composition tile buffers are uploaded to textures.
- Layers with contents buffer
 - Textures from different sources (WebGL, video frame, etc.) are passed to the compositor.
- Layers with a solid color
 - Color information is passed to the compositor to be painted using OpenGL ES directly.



After Skia

- Skia replaced cairo for painting tiled contents.
- No changes when using skia CPU backend.
- Added the GPU rendering mode using Skia ganesh backend.
- Main thread records the layer contents using SkPictureRecorder.
- Worker threads replay the list by drawing the SkPicture into a SkSurface canvas.
- For GPU rendering the SkSurface is a ganesh surface wrapping an external texture.
- On composition tile buffers are already textures.



Compositor

- The compositor iterates the layer tree to paint every layer.
- Layers with tiled backing store
 - Every tile has its own texture that is painted.
- Layers with contents buffer
 - The contents buffer texture is painted.
- Layers with a solid color
 - The color is painted using OpenGL ES.



Texture Mapper (I)

- Compositor was implemented using Texture Mapper.
- Introduced in 2010 by Nokia for the Qt port.
- Later adopted by other ports (GTK, WPE, WinCairo, PlayStation).
- Maintains a collection of shader programs to paint different content.
- Performance is quite good.
- No big changes since then.



Texture Mapper (II)

- Old code mostly unmaintained.
- New features not implemented like blend modes.
- Simple antialiasing implementation.
- Basic clipping support limited to rectilinear and rounded rectangles.
- Compatibility issues with different drivers.



Using Skia for composition

- Main goal is not performance but maintainability.
- The code to iterate the tree, compute transformations, etc. is the same.
- Textures are painted into a Skia canvas surface.
- Fixes compatibility issues with some drivers.
- No need to maintain shader programs anymore.
- Better clipping and antialiasing.
- Initial performance was the same as Texture Mapper or even worse in some cases.



Filters

- Filters were implemented using an intermediate surface like with Texture Mapper.
- Most of the filters can be converted to a simpler SkColorFilter that skia can paint without needed an intermediate surface.
- For those cases the performance improved a lot by not using the intermediate surface.



Masks (I)

- Two types of masked content: image mask and clip path
- Both cases were implemented the same way
- A mask layer was added as a child of the masked layer.
- The masked layer used an intermediate surface.
- The masked layer painted its contents into the intermediate surface.
- Then the mask layer was painted into the intermediate surface in mask mode (like DstIn blend mode)
- Finally the intermediate surface was painted into its parent surface.



Masks (II)

- Clip path:
 - Can be implemented with skia using `SkCanvas::clipPath()`.
 - The mask layer contains the clip path as a property instead.
 - No need to render the mask layer, nor using an intermediate surface at all.
- Image mask:
 - Can be implemented with skia using `SkCanvas::clipShader()`.
 - The mask layer contains the image that is painted once.
 - The masked layer creates `SkShader` using the mask `SkImage`.
 - No need to use an intermediate surface either.



Blend modes

- Blend modes were never implemented in Texture Mapper.
- Supporting blend modes with Skia was a matter of setting the blend mode in the SkPaint when needed.



Batched painting (I)

- Skia keeps an internal queue of tasks so that paint operations on a canvas are queued first.
- When the context is flushed the queue is iterated first to try to merge items and reduce the final amount GL operations.
- This is an improvement in most of the cases, but when there are many items in the queue with very fast GL operations the CPU work to iterate the long queue might be more than just running all GL operations directly.



Batched painting (II)

- Skia has an experimental API that allows (among other things) to do the batching on the user side.
- The skia compositor groups paint operations in batches when possible, so that every batch becomes a single operation in the skia internal queue.
- In the best case every batch also becomes a single GL operation in the end.



Deferred display lists (II)

- Skia GPU requires to create a GrDirectContext for every thread using GL.
- Worker threads create their own GrDirectContext to paint the tiles.
- A fence is used for the synchronization between worker threads and the compositor thread.
- This works but it's not really the recommended way to implement threading with Skia ganesh.
- A deferred display list records draw operations as GL commands to be replayed in a GPU thread.
- Only the compositor thread needs to have a GrDirectContext because workers don't use GL at all.



Deferred display lists (II)

- Main thread keeps recording using SkPicture
- Worker threads paints the recorded SkPicture into a deferred display list recorder canvas
- The compositor replays the deferred display list into a Skia ganes surface used for the tile.
- CPU work is still done in workers, but all GL work is now done in the compositor.
- Performance is neutral in most of the tests, but a bit worse in a few others.



Current situation

- Skia compositor is used by default in WPEWebKit.
- Texture Mapper can still be used by using a runtime setting.
- Deferred display lists can also be disabled using an env var, since it's a new feature and regressed some tests.



Plans

- The runtime setting to enable Texture Mapper will be removed, so it will become a build time option and eventually Texture Mapper will be unsupported.
- Current performance is much better in some tests, neutral in most of the tests and a bit worse in a few tests.
- We will continue working on improving the performance whenever possible.
- Implement per edge antialiasing.
- Switch to vulkan (Skia graphite backend).



Questions



Thanks!



